PROJETO UNICARONA – APLICATIVO DE CARONAS UNIVERSITÁRIAS

TRABALHO FINAL DE ENGENHARIA DE SOFTWARE

1. DEFINIÇÃO DO PROJETO

1.1. Tema do Projeto
UniCarona– Aplicativo de Caronas Universitárias

1.2. Nome do Grupo
Grupo UniCarona

1.3. Participantes

· Hiago Rocha de Jesus
· Fabrício Santana Ramos
· Gabriel Lana Bergamini de Oliveira
· Carlos Daniel Cavalcante Silva
· Rafaela Victoria Pego Bella
· Sofia Aranda Rodrigues

1.4. Lista de Funcionalidades (Backlog Inicial)

· Cadastro de usuários (motoristas e passageiros)
· Criação e busca de caronas disponíveis
· Sistema de localização e filtragem por rota
· Chat interno entre usuários
· Avaliação de motoristas e passageiros
· Histórico de caronas realizadas
· Notificações de confirmação e cancelamento

2. PREMISSAS, RESTRIÇÕES E REQUISITOS

2.1. Premissas

· Usuários possuem e-mail institucional válido
· Motoristas possuem CNH e veículo próprio
· Aplicativo será utilizado em ambiente universitário
· Usuários possuem smartphones compatíveis

2.2. Restrições

· Somente e-mails institucionais são aceitos
· Sistema deve ser gratuito para usuários
· Integração com APIs externas (Google Maps, notificações)
· Compatibilidade com Android e iOS

2.3. Requisitos Funcionais

· RF01: Permitir cadastro e login de usuários
· RF02: Permitir registrar veículo e disponibilidade de rotas
· RF03: Exibir lista de caronas próximas ao usuário
· RF04: Permitir reserva e confirmação de carona
· RF05: Permitir avaliar o motorista ou passageiro após a viagem
· RF06: Exibir notificações e mensagens internas
· RF07: Gerenciar perfil do usuário
· RF08: Consultar histórico de caronas

2.4. Requisitos Não Funcionais

· RNF01: Interface responsiva e intuitiva
· RNF02: Tempo de resposta inferior a 3 segundos
· RNF03: Sistema disponível 24h, com 99% de uptime
· RNF04: Proteção de dados via criptografia
· RNF05: Compatibilidade com Android e iOS
· RNF06: Segurança na autenticação de usuários

3. CASOS DE USO E REGRAS DE NEGÓCIO

3.1. Regras de Negócio

· RN01: Apenas usuários com e-mail institucional podem se cadastrar
· RN02: Um motorista só pode oferecer carona se possuir CNH válida
· RN03: Passageiro deve confirmar presença até 30 minutos antes do horário
· RN04: Avaliações afetam a visibilidade do motorista nas buscas
· RN05: Cancelamentos frequentes reduzem a pontuação do usuário
· RN06: Motorista deve ter veículo cadastrado para oferecer carona
· RN07: Passageiro pode cancelar solicitação até 1 hora antes do horário

3.2. Diagrama de Atores

Atores Identificados:

· Usuário
· Motorista
· Passageiro
· Administrador
· Serviço de Notificações
· Google Maps API

3.3. Descrição dos Atores

Usuário

· Responsabilidade: Realizar cadastro, login, gerenciar informações pessoais, interagir com o sistema como motorista ou passageiro

Motorista

· Responsabilidade: Cadastrar veículo, oferecer caronas, aceitar ou negar solicitações, avaliar passageiros

Passageiro

· Responsabilidade: Buscar caronas disponíveis, solicitar participação em uma carona, avaliar motoristas

Administrador

· Responsabilidade: Gerenciar usuários e caronas, analisar denúncias, emitir relatórios e estatísticas

Serviço de Notificações

· Responsabilidade: Enviar alertas de novas solicitações e atualizações, informar status de caronas em tempo real

Google Maps API

· Responsabilidade: Calcular rotas e distâncias, fornecer geolocalização para o aplicativo

3.4. Diagramas de Casos de Uso

Pacotes de Funcionalidade Identificados:

Pacote 1: Autenticação e Perfil

· Cadastrar-se
· Fazer Login
· Gerenciar Perfil

Pacote 2: Gerenciamento de Caronas

· Cadastrar Veículo
· Oferecer Carona
· Gerenciar Caronas
· Buscar Caronas
· Solicitar Carona
· Aceitar Solicitação
· Negar Solicitação

Pacote 3: Avaliações e Histórico

· Avaliar Motorista
· Avaliar Passageiro
· Consultar Histórico

Pacote 4: Administração e Relatórios

· Gerenciar Usuários
· Gerenciar Caronas e Denúncias
· Emitir Relatórios

Pacote 5: Serviços Externos

· Enviar Notificações Push
· Calcular Rotas e Distâncias

3.5. Especificação dos Casos de Uso

UC01 – Oferecer Carona

· Identificador: UC01
· Nome: Oferecer Carona
· Ator Primário: Motorista
· Atores Secundários: Sistema de Notificações, Google Maps API
· Pré-condições: Usuário autenticado como motorista, veículo cadastrado
· Pós-condições: Carona publicada e visível para passageiros
· Regras de Negócio: RN02, RN04, RN06

Fluxo Principal:

1. Motorista acessa o sistema
2. Seleciona a opção "Oferecer Carona"
3. Informa origem, destino, horário e número de vagas
4. Sistema valida os dados
5. Sistema publica a carona
6. Sistema envia notificação aos passageiros interessados

Fluxo Alternativo:
3a.Caso o veículo não esteja cadastrado, o sistema solicita o registro antes de prosseguir

Fluxo de Exceção:
4a.Dados inválidos - sistema solicita correção e retorna ao passo 3

UC02 – Solicitar Carona

· Identificador: UC02
· Nome: Solicitar Carona
· Ator Primário: Passageiro
· Atores Secundários: Motorista, Sistema de Notificações
· Pré-condições: Usuário autenticado como passageiro
· Pós-condições: Solicitação enviada ao motorista
· Regras de Negócio: RN03, RN07

Fluxo Principal:

1. Passageiro acessa o sistema
2. Pesquisa por caronas compatíveis com sua rota
3. Seleciona uma carona e clica em "Solicitar Carona"
4. Sistema envia solicitação ao motorista
5. Sistema atualiza status da solicitação
6. Passageiro aguarda confirmação

Fluxo Alternativo:
2a.Caso não haja caronas disponíveis, o sistema exibe sugestões de rotas semelhantes
4a.Passageiro pode cancelar solicitação antes da confirmação do motorista

Fluxo de Exceção:
4b.Se a conexão falhar durante o envio da solicitação, o sistema notifica o erro e solicita nova tentativa
6a.Se o motorista recusar, o sistema informa ao passageiro e sugere outras opções disponíveis

UC03 – Gerenciar Perfil

· Identificador: UC03
· Nome: Gerenciar Perfil
· Ator Primário: Usuário
· Pré-condições: Usuário autenticado
· Pós-condições: Perfil atualizado no sistema

Fluxo Principal:

1. Usuário faz login no sistema
2. Seleciona opção "Gerenciar Perfil"
3. Visualiza informações do perfil atual
4. Altera dados desejados (nome, e-mail, telefone, foto, senha, preferências)
5. Confirma alterações
6. Sistema valida dados
7. Sistema atualiza perfil no banco de dados
8. Sistema exibe mensagem de confirmação

4. DIAGRAMA DE CLASSES

Classes Principais Identificadas:

Usuario

· idUsuario: Long
· nome: String
· emailInstitucional: String

· senha: String
· telefone: String
· pontuacao: Double
· fotoPerfil: String
· ativo: Boolean

· autenticar(): Boolean
· atualizarPerfil(): void

Motorista (herda de Usuario)

· cnh: String

· oferecerCarona(): void
· aceitarSolicitacao(): void
· negarSolicitacao(): void

Passageiro (herda de Usuario)

· buscarCaronas(): void
· solicitarCarona(): void
· avaliarMotorista(): void

Administrador (herda de Usuario)

· gerenciarUsuarios(): void
· gerenciarCaronasDenuncias(): void
· emitirRelatorios(): void

Veiculo

· placa: String
· modelo: String
· marca: String
· ano: int
· capacidade: int

· cadastrarVeiculo(): void
· atualizarVeiculo(): void

Carona

· idCarona: Long
· origem: String
· destino: String
· dataHora: DateTime
· vagasDisponiveis: int
· status: String

· publicarCarona(): void
· atualizarCarona(): void
· cancelarCarona(): void

Solicitacao

· idSolicitacao: Long
· status: String

· enviarSolicitacao(): void
· aceitarSolicitacaoDeCarona(): void
· negarSolicitacaoDeCarona(): void

Avaliacao

· idAvaliacao: Long
· nota: int
· comentario: String
· dataAvaliacao: DateTime

· registrarAvaliacao(): void

Historico

· idHistorico: Long
· dataConsulta: DateTime

· consultarHistoricoDeCaronas(): void

Notificacao

· idNotificacao: Long
· tipo: String
· mensagem: String
· dataEnvio: DateTime

· enviarNotificacaoPush(): void

Rota

· distancia: Double
· duracao: Double

· calcularRota(): void

Relacionamentos entre Classes:

· Herança: Usuario → Motorista / Passageiro / Administrador
· Associação:

  · Motorista → Veiculo (1:N)
  · Motorista → Carona (1:N)
  · Passageiro → Solicitacao (1:N)
  · Carona → Solicitacao (1:N)
  · Usuario → Avaliacao (1:N)
· Composição: Usuario → Historico

5. DIAGRAMAS COMPORTAMENTAIS

5.1. Diagramas de Estados

Diagrama de Estados 1 – Oferecer Carona

Estados:

· Iniciar
· InserirDadosVeiculo
· InformarCarona
· ConfirmarOferta
· ValidarDados
· Concluído
· Erro
· Corrigir

Transições:

· Iniciar → InserirDadosVeiculo (selecionar "Oferecer Carona")
· InserirDadosVeiculo → InformarCarona (preencher dados do veículo)
· InformarCarona → ConfirmarOferta (informar origem, destino, data, horário e vagas)
· ConfirmarOferta → ValidarDados (clicar "Confirmar")
· ValidarDados → Concluído (dados válidos)
· ValidarDados → Erro (dados inválidos)
· Erro → Corrigir (corrigir dados)
· Corrigir → ValidarDados

Diagrama de Estados 2 – Solicitar Carona

Estados:

· AcessarSistema
· BuscarCaronas
· SelecionarCarona
· EnviarSolicitacao
· AguardarResposta
· Aceita
· Negada
· ConfirmarReserva

Transições:

· AcessarSistema → BuscarCaronas (fazer login)
· BuscarCaronas → SelecionarCarona (visualizar lista)
· SelecionarCarona → EnviarSolicitacao (clicar "Solicitar Carona")
· EnviarSolicitacao → AguardarResposta (solicitação enviada)
· AguardarResposta → Aceita (motorista aceita)
· AguardarResposta → Negada (motorista nega)
· Aceita → ConfirmarReserva (confirmar reserva da vaga)

Diagrama de Estados 3 – Avaliar Usuário

Estados:

· AguardandoAvaliacao
· PreencherAvaliacao
· ConfirmarAvaliacao
· SalvarAvaliacao
· Concluído
· Erro
· TentarNovamente

5.2. Diagramas de Atividades

Diagrama de Atividade 1 – Oferecer Carona

1. Fazer login no sistema
2. Selecionar opção "Oferecer Carona"
3. Inserir dados do veículo
4. Informar origem, destino, data e horário da carona
5. Definir número de vagas disponíveis
6. Confirmar oferta de carona
7. Sistema valida dados
8. Sistema registra carona
9. Sistema envia notificação aos passageiros interessados

Diagrama de Atividade 2 – Solicitar Carona

1. Fazer login no sistema
2. Buscar caronas disponíveis (origem, destino, data, horário)
3. Selecionar carona desejada
4. Clicar em "Solicitar Carona"
5. Sistema envia solicitação ao motorista
6. Sistema atualiza status da solicitação
7. Receber notificação de aceite ou recusa

8. [Aceita] Confirmar reserva da vaga
9. [Negada] Sugerir outras caronas disponíveis

Diagrama de Atividade 3 – Avaliar Usuário

1. Fazer login no sistema
2. Após conclusão da carona, receber solicitação de avaliação
3. Selecionar nota (1 a 5 estrelas)
4. Inserir comentário opcional
5. Confirmar avaliação
6. Sistema registra avaliação
7. Sistema atualiza pontuação do usuário avaliado
8. Sistema exibe mensagem de confirmação

Diagrama de Atividade 4 – Aceitar Solicitação de Carona

1. Fazer login no sistema
2. Acessar solicitações de carona recebidas
3. Selecionar solicitação a analisar
4. Clicar em "Aceitar Solicitação"
5. Sistema confirma aceitação
6. Sistema reserva vaga para o passageiro
7. Sistema envia notificação ao passageiro
8. Sistema atualiza status da carona

Diagrama de Atividade 5 – Consultar Histórico de Caronas

1. Fazer login no sistema
2. Selecionar opção "Histórico de Caronas"
3. Sistema exibe lista de caronas anteriores (data, horário, motorista/passageiro, status, avaliação)
4. [Opcional] Selecionar carona específica para detalhes
5. Sistema apresenta detalhes completos
6. [Opcional] Aplicar filtros (período, motorista, passageiro)
7. [Opcional] Exportar histórico em PDF ou Excel

5.3. Diagramas de Sequência

Diagrama de Sequência 1 – Oferecer Carona

Participantes: Motorista, Sistema (Boundary), GestorCaronas (Controller), Carona (Entity), Veiculo (Entity)

Fluxo de Mensagens:

1. Motorista → Sistema: Acessa "Oferecer Carona"
2. Sistema → GestorCaronas: Solicita dados do veículo

3. GestorCaronas → Veiculo: Valida dados do veículo
4. Veiculo → GestorCaronas: Retorna dados válidos
5. GestorCaronas → Sistema: Solicita dados da carona
6. Sistema → Motorista: Exibe formulário para dados da carona
7. Motorista → Sistema: Informa origem, destino, horário, vagas
8. Sistema → GestorCaronas: Envia dados da carona
9. GestorCaronas → Carona: Cria nova carona
10. Carona → GestorCaronas: Confirma criação
11. GestorCaronas → Sistema: Confirma publicação
12. Sistema → Motorista: Exibe confirmação

Diagrama de Sequência 2 – Solicitar Carona

Participantes: Passageiro, Sistema (Boundary), GestorCaronas (Controller), Carona (Entity), Motorista

Fluxo de Mensagens:

1. Passageiro → Sistema: Busca caronas disponíveis
2. Sistema → GestorCaronas: Solicita lista de caronas
3. GestorCaronas → Carona: Consulta caronas disponíveis
4. Carona → GestorCaronas: Retorna lista
5. GestorCaronas → Sistema: Envia lista
6. Sistema → Passageiro: Exibe caronas disponíveis
7. Passageiro → Sistema: Seleciona carona e clica "Solicitar"
8. Sistema → GestorCaronas: Envia solicitação
9. GestorCaronas → Motorista: Notifica sobre solicitação
10. Motorista → GestorCaronas: Aceita solicitação
11. GestorCaronas → Sistema: Confirma aceitação
12. Sistema → Passageiro: Notifica confirmação

Diagrama de Sequência 3 – Aceitar Solicitação

Participantes: Motorista, Sistema (Boundary), GestorSolicitacoes (Controller), Solicitacao (Entity), Passageiro

Diagrama de Sequência 4 – Negar Solicitação

Participantes: Motorista, Sistema (Boundary), GestorSolicitacoes (Controller), Solicitacao (Entity), Passageiro

Diagrama de Sequência 5 – Avaliar Usuário

Participantes: Usuário, Sistema (Boundary), GestorAvaliacoes (Controller), Avaliacao (Entity), UsuarioAvaliado (Entity)

6. DESIGN PATTERNS UTILIZADOS

6.1. Padrão de Criação – Singleton

· Classe: NotificacaoSingleton
· Propósito: Garantir que exista apenas uma instância do serviço de notificações
· Implementação:
  · instancia: NotificacaoSingleton
  · getInstancia(): NotificacaoSingleton
  · enviarNotificacao(usuario, mensagem): void

6.2. Padrão Estrutural – Facade

· Classe: SistemaCaronaFacade
· Propósito: Fornecer uma interface simplificada para o sistema complexo de caronas
· Métodos:
  · oferecerCarona(motorista, carona): void
  · solicitarCarona(passageiro, carona): void
  · aceitarSolicitacao(motorista, solicitacao): void
  · negarSolicitacao(motorista, solicitacao): void
  · consultarHistorico(usuario): List<Carona>

6.3. Padrão Comportamental – Observer

· Classes: Observer, PassageiroObserver, MotoristaObserver
· Propósito: Notificar automaticamente os usuários sobre mudanças de estado
· Implementação:
  · atualizar(mensagem): void

7. CONCLUSÃO E COMENTÁRIOS

O desenvolvimento do UniCarona reforçou o entendimento prático da UML e da Engenharia de Software. A aplicação dos conceitos teóricos possibilitou a criação de um projeto funcional, coerente e sustentável. O trabalho em equipe aprimorou a comunicação, a organização e a divisão de tarefas. Os diagramas facilitaram a visualização das interações e dependências entre os componentes do sistema. A experiência foi fundamental para consolidar o que é ensinado em sala de aula, demonstrando a importância do planejamento e da documentação no desenvolvimento de software.

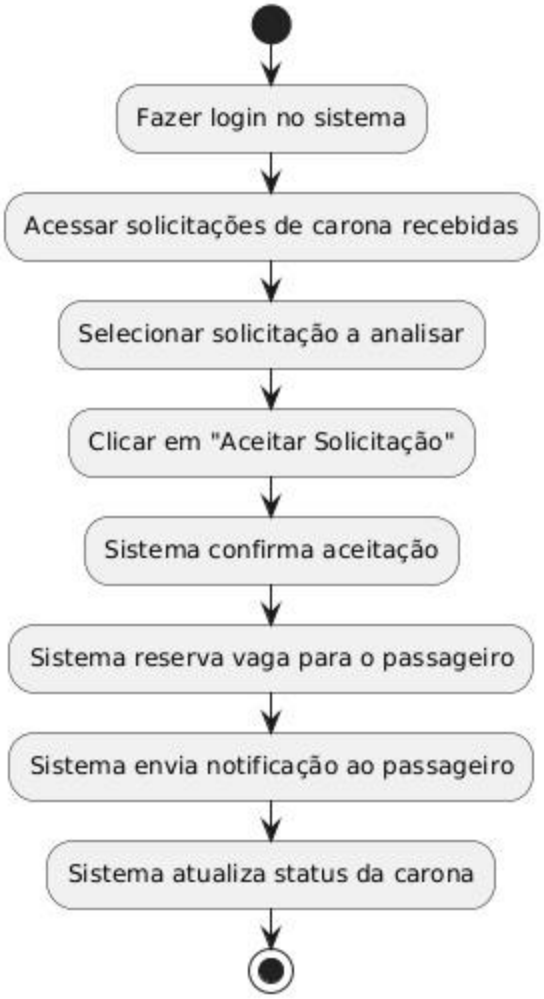
8. REFERÊNCIAS

· Material didático da disciplina de Engenharia de Software
· Documentação oficial da UML
· Guias de melhores práticas em desenvolvimento de software

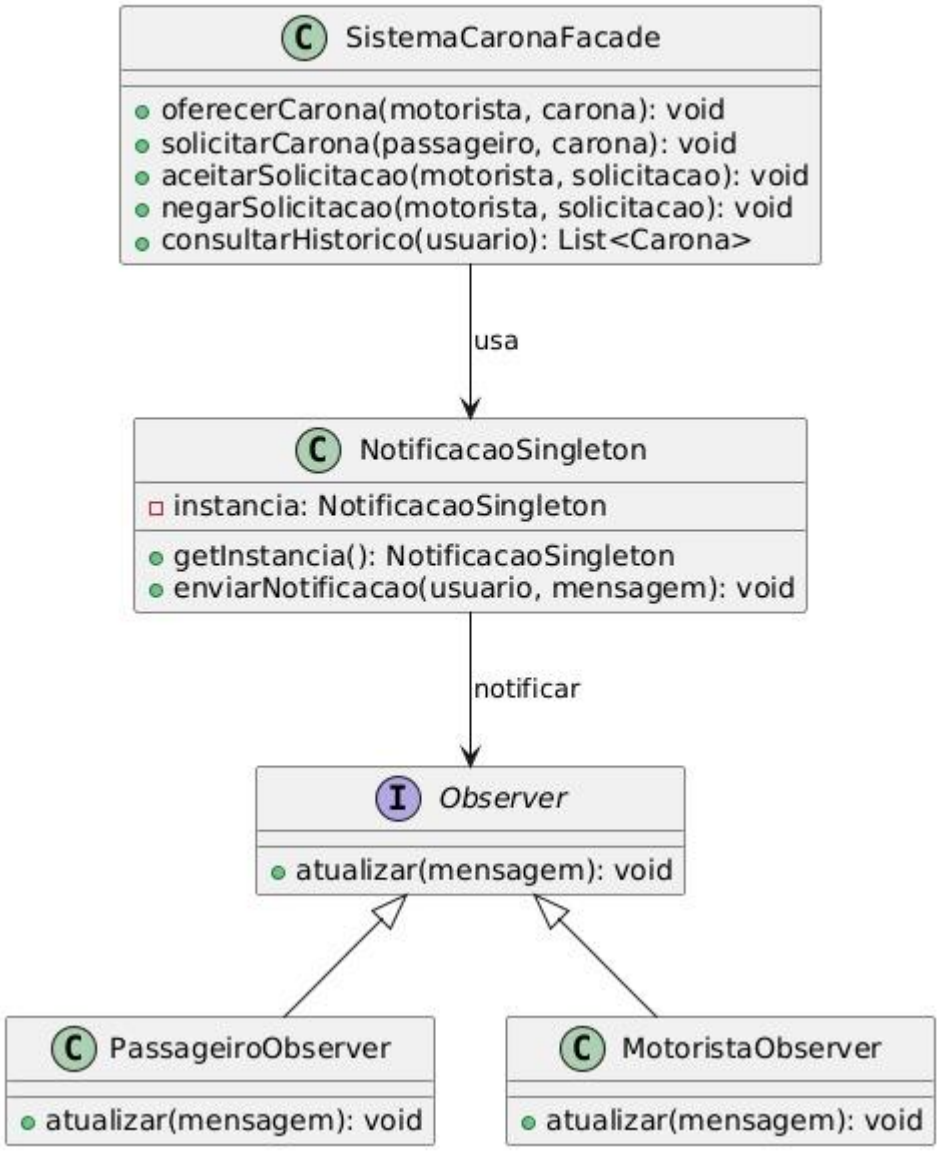
Imagens dos Diagramas Gerais

Espaço para inserção dos diagramas:

Diagrama de Atividade - Aceitar Solicitação de Carona



Design Patterns - UniCarona



UniCarona - Diagrama de Classes

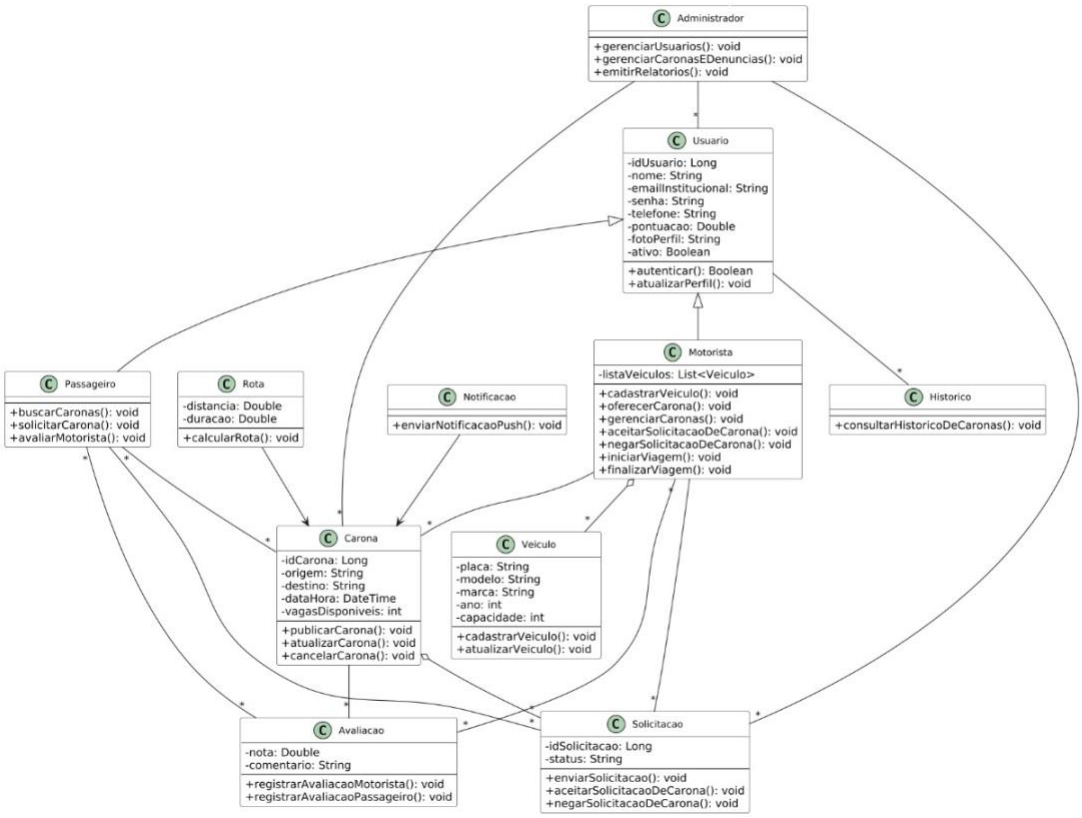


Diagrama de Casos de Uso - UniCarona

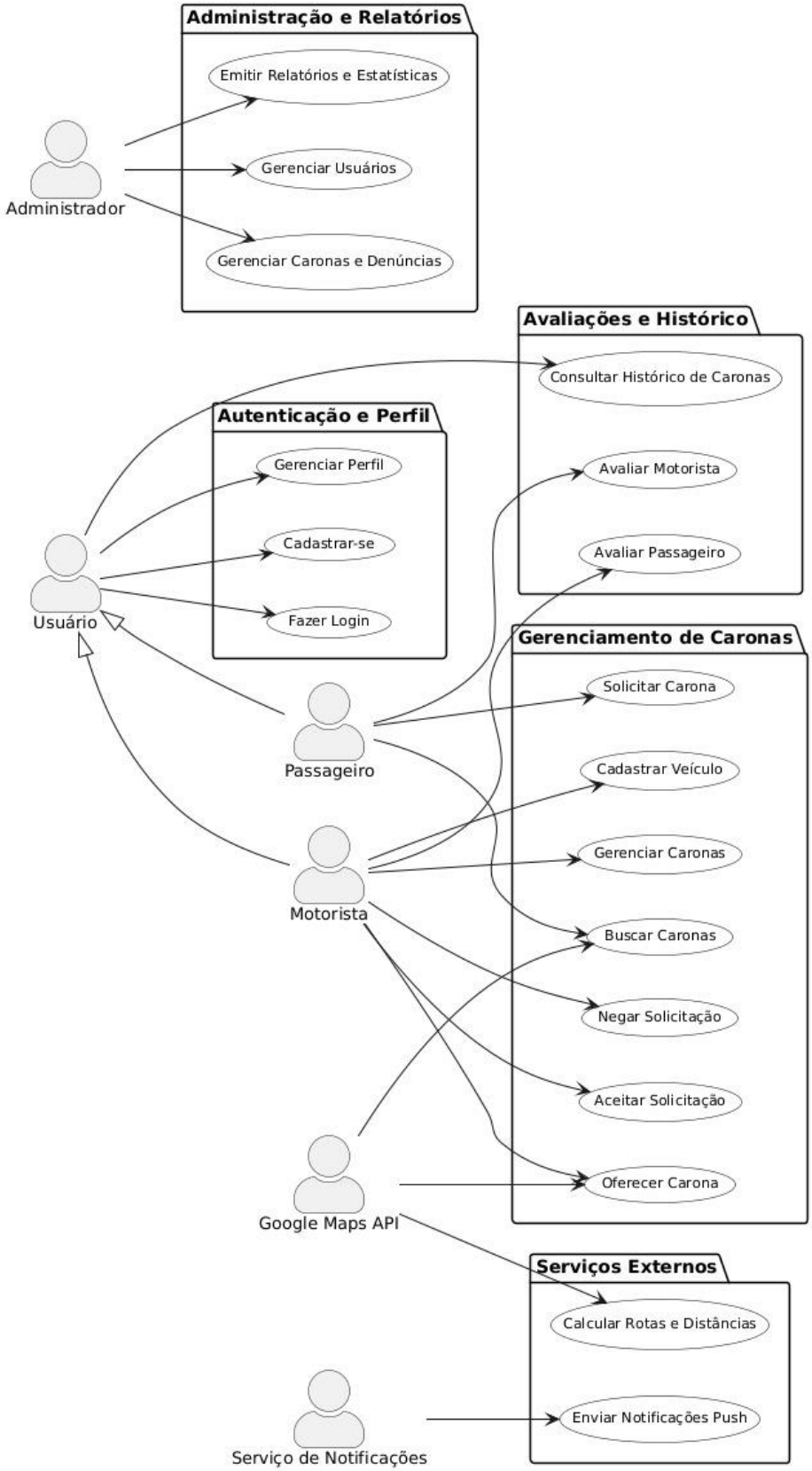
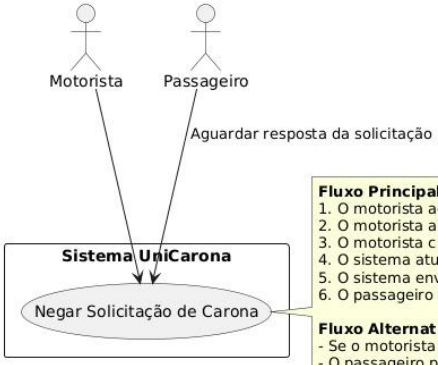


Diagrama de Caso de Uso - Negar Solicitação de Carona



Fluxo Principal:
1. O motorista acessa o sistema e visualiza as solicitações de carona recebidas.
2. O motorista analisa a solicitação do passageiro.
3. O motorista clica em "Negar Solicitação".
4. O sistema atualiza o status da solicitação para "Negada".
5. O sistema envia notificação ao passageiro informando que a solicitação foi recusada.
6. O passageiro pode visualizar outras caronas disponíveis e realizar nova solicitação.

Fluxo Alternativo:
- Se o motorista quiser alterar informações da carona antes de negar, ele pode acessar "Gerenciar Caronas".
- O passageiro pode cancelar a solicitação antes da resposta do motorista, encerrando o processo.

Fluxo de Exceção:
- Se houver falha de comunicação com o servidor, o sistema exibe erro e solicita nova tentativa.
- Se a solicitação já tiver sido aceita por outro passageiro ou motorista, o sistema informa o conflito.